Chapter 1

Overview and Architecture

1.1 What OpenSG-2.0 is

OpenSG-2.0 is an implementation of Mechanics of Structure Genome (MSG) homogenization written in JAX. You give it a *structure gene* (SG) — the smallest piece of a structure that carries its heterogeneity — and it returns the macroscopic constitutive law of the structure that SG builds: a Timoshenko beam 6×6, a plate ABD (or the shear-refined 8×8), or an equivalent 3-D solid 6×6. It also runs the same calculation backwards (*dehomogenization*), rebuilding the pointwise 3-D stress and strain inside the SG from macroscopic beam or plate loads.

The repository is organised in four layers. Only the top layer knows where files live on disk.

Table 1.1: The four layers of OpenSG-2.0. Paths and `main` blocks exist only in the top row.

Layer	Package	Role
user	`examples/`	YAML in → `.out` out; owns all paths
msg-shell	`src/opensg_shell`	Reissner–Mindlin shell SGs: ring (cross-section), aperiodic segment, 3-D shell cell
msg-solid	`src/opensg_solid`	solid SGs: plate ABD, beam KKT, 3-D solid law
FE core	`src/fe_jax`	JAX FE architecture: basis/quadrature, periodic assembly map, jitted element kernels, element-by-element (EBE) Chebyshev-CG, sparse direct

1.1.1 The universal contract

Every capability in both engines obeys the same contract: **a YAML file (or a SwiftComp `.sc` file) goes in, and a timed `.out` file in the SwiftComp `.K` layout comes back.** There is exactly one writer for that file, `opensg_solid.sg_homo.write_sc_K`, and it is called from inside the solver source, not from the example scripts, so no run can forget it. A minimal session looks like this:

```
from opensg_shell import build_rm_bundle

B = build_rm_bundle("iea_s10_shell.yaml")   # writes iea_s10_shell_Timo.out
B["Timo"]                                    # the Timoshenko 6x6 as a numpy array
```

Listing 1.1: The whole contract: the .out file is written as a side effect of the call.

The file that comes back always has the same shape: an `OpenSG` banner naming the model and the SG measure ω on the first line, the effective stiffness matrix, its inverse (the compliance), an engineering-constants block for 3-D laws only, and `Time taken` on the last line. Numbers are printed in a 19-character right-justified Fortran field (`%.7E` mantissa, signed three-digit exponent) so the file drops straight into existing SwiftComp comparison tooling:

```
OpenSG msg-shell beam model [ext sh2 sh3 twist bend2 bend3]

The Effective Timoshenko Stiffness Matrix
-----------------------------------------
    <6 numbers per row>
...
The Effective Timoshenko Compliance Matrix
-----------------------------------------
...
Time taken: 12.34 sec
```

The suffix on the output file names the model, so two different routes run on the same input YAML do not silently overwrite each other (Table 1.2).

Table 1.2: Output file names. There is one file per model; the `_Timo` vs `_C3D` suffix disambiguates when two routes can read the same YAML.

Model	Entry function	Output file
msg-solid beam / plate / 3-D	`plate_homo_2d`	`<base>.out`
msg-shell beam (Timoshenko)	`build_rm_bundle`	`<yaml>_Timo.out`, `<yaml>_ABDG.out`
msg-shell aperiodic segment	`segment_timo_from_3dyaml`	`<yaml>_Timo.out`
msg-shell solid props, cross-section SG	`build_solid_bundle`	`<yaml>_C3D.out`
msg-shell 3-D shell SG	`shell_sg3d`	`<yaml>_C3D.out`

1.1.2 The cores hold no paths

Neither `opensg_shell` nor `opensg_solid` contains a `main` block, a hard-coded directory, or an assumption about where you run from. Every path decision lives in the example script, which names its input file and calls exactly one entry function on it. That is why the tutorials in the later chapters can be reduced to two lines of user input and one call: to run a different cross-section you change the string, not the library.

Two pieces of process-level setup do live in the package `__init__` files, and both matter to you:

- **Double precision is load-bearing.** Both packages execute `jax.config.update("jax_enable_x64", True)` at import. Without it JAX silently degrades to float32 and the digits of the 6×6 are wrong. Import the package (`import opensg_shell`) before you import anything else that touches JAX.
- **A persistent compile cache.** `opensg_solid/__init__.py` points JAX's compilation cache at `$JAX_COMPILATION_CACHE_DIR`, defaulting to `~/.cache/opensg_jax`, with `jax_persistent_cache_min_compile_time_secs` set to 0. A cold run pays each XLA compile once; warm runs load the binaries back. The cache is wrapped in a `try/except` — it is an

optimisation, and the engine must run without it. The same file honours `$FE_JAX_CORE`, which prepends an external `fe_jax` checkout to `sys.path` to override the bundled copy at `src/fe_jax`.

1.2 The theory in one page

Every pipeline in this manual is the same variational statement, specialised. It is worth reading this section once, because the symbols used here are the variable names used in the source: `Dee`, `Dhe`, `Dhh`, `V0`, `V1`, `omega`.

1.2.1 The structure gene and the strain split

A structure gene is the smallest piece of the structure that carries the heterogeneity: a laminate stacking sequence through the thickness (a 1-D SG), a cross-section contour or a 2-D unit cell (a 2-D SG), a lattice or TPMS unit cell (a 3-D SG). The displacement field inside the SG is split into a macroscopic part driven by the macro strains $\bar{\varepsilon}$ and a fluctuation (warping) field w. In strain form,

$$\varepsilon \;=\; \Gamma_e\,\bar{\varepsilon} \;+\; \Gamma_h\,w, \tag{1.1}$$

where Γ_e is the operator that embeds the macro strains into the SG and Γ_h is the SG's own strain operator acting on the warping. The two are not independent: Γ_e is Γ_h evaluated on the macro embedding, a consistency requirement written down in `Rules/gamma_e_gamma_h_consistency.md` and enforced by building both from the same element kernel.

1.2.2 The quadratic form

Substituting (1.1) into the strain energy gives the quadratic form the code assembles, in exactly the block names used throughout the source:

$$2U \;=\; \bar{\varepsilon}^T\mathbf{D}_{ee}\,\bar{\varepsilon} \;+\; 2\,w^T\mathbf{D}_{he}\,\bar{\varepsilon} \;+\; w^T\mathbf{D}_{hh}\,w. \tag{1.2}$$

Read it as three integrals over the SG: $\mathbf{D}_{ee}$ is the macro-macro block (what you would get with no warping at all, the rule-of-mixtures answer), $\mathbf{D}_{hh}$ is the SG stiffness seen by the warping alone, and $\mathbf{D}_{he}$ couples them.

1.2.3 The zeroth-order solve

Minimising (1.2) over w, subject to the SG's boundary treatment (periodicity, Dirichlet boundary data, or a rigid-body constraint that removes the null space) gives one linear system with as many right-hand sides as there are macro strains,

$$\mathbf{D}_{hh}\,\mathbf{V}_0 \;=\; -\,\mathbf{D}_{he}, \tag{1.3}$$

and the effective law follows by substituting the minimiser back:

$$\mathbf{D}_{\text{eff}} \;=\; \mathbf{D}_{ee} + \mathbf{V}_0^T\mathbf{D}_{he}, \qquad \mathbf{C} \;=\; \mathbf{D}_{\text{eff}}/\omega. \tag{1.4}$$

Here ω is the SG measure that stays in the model — the thickness of a 1-D laminate SG, the area of a cross-section SG, the volume of a 3-D solid cell, the perimeter (or, in three dimensions,

the midsurface area) of a shell SG. The banner of every `.out` states ω because a bare matrix is ambiguous: a whole-matrix ratio that is uniform across all entries (exactly 0.5000, say) is a normalisation difference, not a physics discrepancy.

$\mathbf{V}_0$ is stored, not discarded. Its columns are the warping fields per unit macro strain, and they are what the dehomogenization reads back.

Equation (1.4) is the **zeroth-order** answer: the plate ABD, the Euler–Bernoulli 4×4, the equivalent 3-D solid law. It contains no transverse shear, because the classical macro models it corresponds to have none.

1.2.4 The first-order step: transverse shear

Recovering transverse shear — the Reissner–Mindlin G block of a plate, the GA_{12} and GA_{13} entries of a Timoshenko 6×6 — takes a **first-order** step: a second solve for $\mathbf{V}_1$ against a right-hand side built from $\mathbf{V}_0$, followed by an energy transformation. Two functions do this, and they are shared by every beam route in the repository:

- `prepare_v1_rhs(V0, Dhl, Dll, Dle_dense, Psi, Dc)` forms the first-order right-hand side and the auxiliary blocks $\mathbf{V}_0^T\mathbf{D}_{ll}\mathbf{V}_0$, $\mathbf{D}_{hl}\mathbf{V}_0$ and $\mathbf{D}_{hl}^T\mathbf{V}_0 + \mathbf{D}_{le}$, projecting the right-hand side onto the range of $\mathbf{D}_{hh}$ so the singular system stays consistent.
- `finalize_v1_and_compute_deff(V1s_raw, V0, D_eff, V0DllV0, DhlV0, DhlTV0Dle, Psi, Dc)` projects $\mathbf{V}_1$, symmetrises $\mathbf{D}_{\text{eff}}$, and performs the generalized-Timoshenko transformation of the energy blocks (A, B, C, D) into the sorted 6×6.

Both live in `opensg_shell/fe_jax/msg_solver.py` and are `@jax.jit`-decorated. The output ordering of the 6×6 is `[EA, GA12, GA13, GJ, EI2, EI3]`, i.e. the strain measures $[\varepsilon_{11}\ \gamma_{12}\ \gamma_{13}\ \kappa_1\ \kappa_2\ \kappa_3]$, which is the VABS diagonal order.

The symmetrisation inside `finalize_v1_and_compute_deff` is not cosmetic. $\mathbf{D}_{\text{eff}}$ is a Hessian and therefore symmetric by construction, but a Dirichlet-constrained tapered-segment solve produces a small (2 to 6 percent) antisymmetric artifact, because with $\mathbf{V}_0$ pinned at the end rings $\mathbf{D}_{ee} + \mathbf{V}_0^T\mathbf{D}_{he}$ is no longer the exact symmetric Schur complement. Left alone, that artifact propagates through the inverse into the antisymmetric shear–bending coupling (C_{26}, C_{35}) and inflates it by up to 78 percent on anisotropic tapers while leaving the symmetric terms untouched. Enforcing the invariant is a no-op for a prismatic SG.

1.2.5 One factorization for two solves

Equations (1.3) and the $\mathbf{V}_1$ system share the same operator. Every beam route therefore factorizes once and back-substitutes twice. In `sg_homo.ring_indep` the augmented KKT matrix

$$\begin{bmatrix} \mathbf{D}_{hh} & \mathbf{D}_c \\ \mathbf{D}_c^T & \mathbf{0} \end{bmatrix} \begin{Bmatrix} \mathbf{V} \\ \lambda \end{Bmatrix} = \begin{Bmatrix} \mathbf{R} \\ \mathbf{0} \end{Bmatrix}$$

is built once, passed to `scipy.linalg.lu_factor`, and then `lu_solve` is called with $\mathbf{R} = -\mathbf{D}_{he}$ for $\mathbf{V}_0$ and with the `prepare_v1_rhs` output for $\mathbf{V}_1$. In the tapered-segment route the same idea appears as `sg_assembly.dirichlet_factor` followed by two `dirichlet_solve_fac` calls; the second call falls back to a fresh factorization only if the $\mathbf{V}_1$ Dirichlet set differs from the $\mathbf{V}_0$ one. In the msg-solid beam engine the equivalent is `solve_fluctuation_field`, which returns the assembled KKT matrix alongside $\mathbf{V}_0$ precisely so the caller can reuse it; there the constraint rows are scaled by 10^8 before assembly to keep the saddle-point system well conditioned, and empty rows are given a unit diagonal with a zeroed right-hand side.

1.2.6 Dehomogenization: running the chain backwards

Given macro strains (or macro forces, converted through $\bar{\varepsilon} = \mathbf{C}^{-1}\mathbf{F}$), the warping is rebuilt from the stored $\mathbf{V}_0$ and $\mathbf{V}_1$, equation (1.1) is evaluated pointwise, and the material law of each element turns that strain into 3-D stress. Nothing is re-solved: homogenize once, recover as often as you like. For the shell engine this is a *two-step* recovery — step 1 rebuilds the six shell strains plus the two wall transverse shears $[\varepsilon_{11}\ \varepsilon_{22}\ 2\varepsilon_{12}\ \kappa_{11}\ \kappa_{22}\ 2\kappa_{12}]$ and $[2\gamma_{13}\ 2\gamma_{23}]$ on the ring, and step 2 pushes them through the through-thickness plate SG to get ply-level 3-D stress. The entry points are `opensg_shell.stress_at_points` and `opensg_shell.disp_at_points` for the shell engine, and `opensg_solid.sg_dehom.dehom_fields` / `plate_dehom_2d` for the solid engine.

1.3 Module map

Both packages are split file-per-concern with the same naming, so a routine you know in one engine is in the same-named file in the other.

The shared FE core `src/fe_jax` is not an OpenSG concern but a general JAX finite-element architecture. The pieces the two engines actually import are `basis_quadrature` (`FiniteElementType`, `get_quadrature`, `eval_basis_and_derivatives`, all backed by basix), `setup.mesh_to_jax` and `setup.mesh_to_periodic_sparse_assembly_map`, `solve_cg` (a conjugate-gradient solver that returns convergence information), `linear_elasticity`, and `profiling`.

1.4 The pipelines

Four routes are described here in the order a new user meets them. Each is a sequence of calls that the entry function makes on your behalf; you only ever call the entry function. Figure 1.1 puts all four side by side: one input file, one entry function, one `.out` file per branch.

1.4.1 msg-shell: 1-D ring → Timoshenko 6×6

This is the composite blade cross-section route. The input is a 1-D shell SG YAML: nodes and two-node elements tracing the wall contour, `elementOrientations`, `sections` carrying a layup per element set, and `materials`. The entry function is `build_rm_bundle(shell_yaml, ref=None, shear="mitc4_g23", g_source="msg")`, and it proceeds as follows.

1. `sg_mesh.load_ring_ref` reads the YAML and returns the contour arrays — nodal coordinates `rx`, connectivity `cells`, the per-element section index `rsub`, the element normals `re3`, the axial index `ax` and the two cross-section indices `cross`. The laminate reference surface comes from the YAML's own `reference` field (absent → `center`, the mid-surface), so homogenization and recovery cannot disagree; pass `ref` explicitly only to override. The reference is converted once into a thickness fraction `frac` (`center` → 0.5, `oml` → 0.0, `oml_flip` and `iml` → 1.0) and that single number then drives the ring laminate reference, the plate-SG z reference, the emitted ABD and the recovery depth conversion.

2. `sg_materials._material_by_section` builds, per section, the wall plate ABD 6×6 at the chosen reference and the 2×2 transverse-shear G.

3. With the default `g_source="msg"`, the wall G of every section is replaced by `rm_plate_msg`, the MSG (Yu-2002 least-squares) through-thickness solve in `opensg_solid.rm_plate_1D`, run

Table 1.3: `src/opensg_shell` — the MSG shell engine.

File	Concern
`sg_mesh.py`	SG input loaders and mesh utilities: 1-D ring YAML loaders (`load_ring`, `load_ring_ref`) with explicit laminate-reference conventions; 3-D segment loading and topological boundary extraction (`extract`) into solver `.npz` bundles; orientation-frame numeric reports and PNG prechecks.
`sg_materials.py`	Section laws: the per-section wall plate ABD 6×6 plus the 2×2 transverse-shear G (`_material_by_section`), and the per-station wall-law emitter `emit_station_abd` / `load_station_abd` that writes one windIO-style YAML of 8×8 Reissner–Mindlin plate laws per cross-section.
`sg_assembly.py`	Shell element operators and assembly: the production 6-DOF drilling element, the 5-DOF general taper element, the prismatic MITC4 element, MITC assumed-strain tying, the `compute_k22` / `compute_kg` initial-curvature corrections, the Dirichlet factorizations (`dirichlet_factor`, `dirichlet_solve_fac`), and the six-block MSG energy assembly (`assemble_segment_indep`, `assemble_constraint`).
`sg_periodicity.py`	Periodic local-to-global sparse assembly map for a shell mesh, 6 DOF per node $(w_1, w_2, w_3, \omega_1, \omega_2, \omega_3)$ instead of the solid engine's three translations. The tie is a plain index map; no transformation matrix is formed.
`sg_homo.py`	Homogenization drivers: `ring_indep` (ring $\rightarrow$ Timoshenko 6×6), `build_rm_bundle` (the packaged bundle the dehom consumes), `ring_solid` / `build_solid_bundle` (cross-section $\rightarrow$ equivalent 3-D solid), `shell_sg3d` (3-D shell SG $\rightarrow$ `C3D`), `segment_timo_from_3dyaml` (aperiodic / tapered segment), `elastic_constants`.
`sg_dehom.py`	Reissner–Mindlin two-step dehomogenization and recovery: `stress_at_points`, `disp_at_points`, built on the same element operators as the homogenization so it is the energy-consistent adjoint.
`sg_junction.py`	Level-2 junction law: local corner and micro-cell solves (`corner_micro_law` for L/T/X topologies, `microcell_law` for a periodic mini-lattice) that correct the shell lattice for sub-shell junction physics.
`fe_jax/`	The OpenSG-authored `msg_*` kernels: `msg_materials` (ABD and 6×6 tables, reference shifts), `msg_transverse_shear` (wall G, the 8×8 plate law), `msg_mesh`, `msg_rm`/`msg_rm_timo` (RM operators and the 5-DOF Timoshenko assembly), `msg_solver` (KKT solve, $\mathbf{V}_1$ RHS, Timoshenko finalization), `msg_hermite` (the Hermite C^1 Kirchhoff–Love thin-walled pipeline), `msg_dehom`, `orient_plot`.
`helper/`	Format converters (`ff_to_yaml.py`).

Table 1.4: `src/opensg_solid` — the general structure-gene engine.

File	Concern
`sg_materials.py`	The 6×6 stiffness tables, per material and per element: `build_material_C` and `get_heterogeneous_C_matrix`, plus the in-plane rotation. Voigt order everywhere is SwiftComp (xx, yy, zz, yz, xz, xy).
`sg_mesh.py`	SG input parsing and mesh utilities: `load_sg_input` (a SwiftComp `.sc` or the helper's YAML), `_cell_basis` (SG dimension plus nodes-per-element $\rightarrow$ basix cell type and basis degree), and `plot_sg_mesh`, which draws the actual mesh coloured by material.
`sg_assembly.py`	Element kernels, periodic assembly, preconditioners and linear solvers: `calculate_RHS_and_Ke_batch_periodic`, `compress_periodic_cells_jax`, `compute_block_inv_diag`, `apply_block_precond`, `apply_chebyshev_precond`, `ebe_jacobian_product_periodic`, `estimate_max_eigenvalue`, `_sparse_direct_solve`, `solve_fluctuation_field`, `plate_ladder_element_blocks`.
`sg_homo.py`	Homogenization drivers and the one public entry `plate_homo_2d`: the periodic CG pipeline (`full_homogenization_pipeline`, `_homo_direct`) for `n_model` 2 and 3, the beam KKT path (`_beam_homo_kkt`) for `n_model` 1, the Reissner–Mindlin first-order plate ladder (`plate_shear_ladder`), and the single output writer `write_sc_K`.
`sg_dehom.py`	Dehomogenization: the Gauss-point recovery kernel for `n_model` 2 and 3, the generalized-Timoshenko beam recovery chain for `n_model` 1, the public `dehom_fields` and `plate_dehom_2d`, and the Gauss exports (`export_gauss`). The batched recovery is wrapped in a module-level `jit` so repeated calls in one process do not re-trace.
`rm_plate_1D/`	The 1-D through-thickness plate stack used by both engines: `segment_plate.plate_sg_yaml` (build the through-thickness SG from a stacking sequence), `msg_rm_plate.rm_plate_msg` (the 8×8 ABDG and the warping ladders), `rm_dehom` (ply-level recovery), `rm_homo`, `cli_rm_plate`.

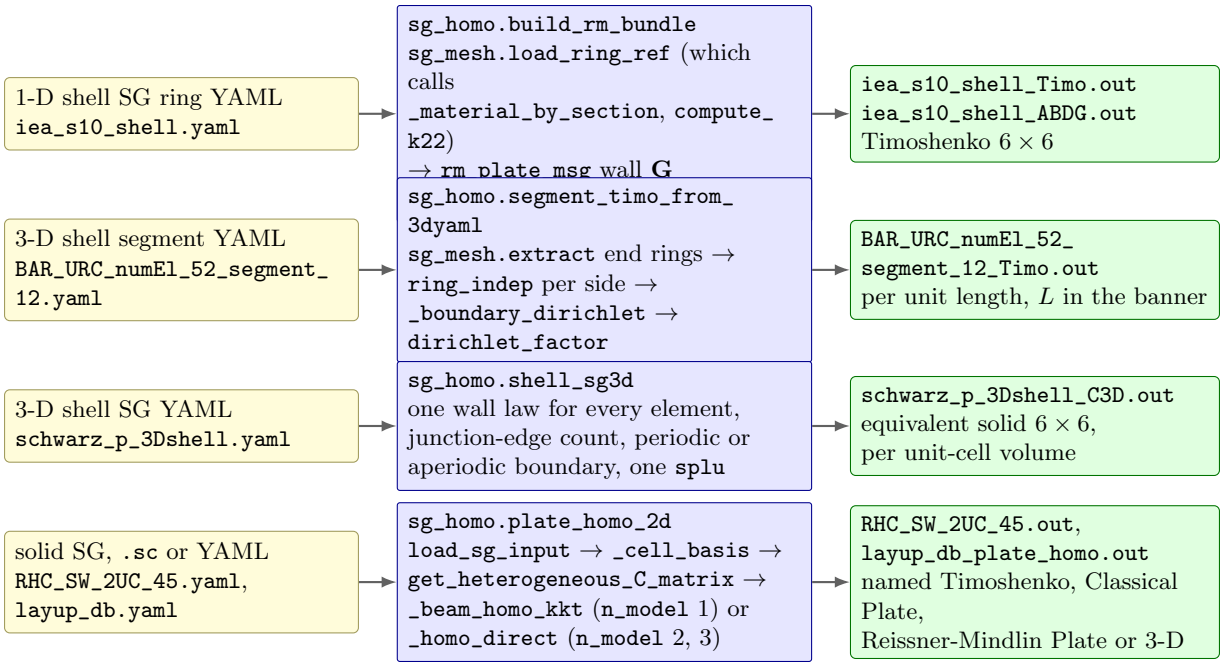

Figure 1.1: The four pipelines, one row each: the three msg-shell routes and the msg-solid driver whose `n_model` argument selects the macro model. Yellow is the file you edit, blue the entry function with the calls it makes in order, green the `.out` file it leaves behind. Every one of the four green nodes is written by the same function, `opensg_solid.sg_homo.write_sc_K`, which is why the four files share one layout.

at the same `fraction=frac`. The alternative is `g_source="whitney"`, the complementary-energy shear-flow value. The per-section laws are written to `<yaml>_ABDG.out`.

4. `sg_assembly.compute_k22` computes the hoop curvature per edge, the initial-curvature correction that makes a curved wall stiffer than a faceted approximation of it.

5. `sg_homo.ring_indep` assembles the ring as a one-element-deep strip of quads: the 6-DOF drilling element $(w_1, w_2, w_3, \theta_1, \theta_2, \omega_3)$ with an element-wise Lagrange multiplier enforcing the drilling constraint, and MITC tying of γ_{23} only. (Under span invariance γ_{13} is algebraic in the directors and carries no fluctuation gradient, so only γ_{23} pairs a differentiated displacement with a rotation.) The six energy blocks `Dhh, Dhe, Dee, Dhl, Dll, Dle` come out of `assemble_segment_indep` and the drilling rows `Gc, Gl, Ge` out of `assemble_constraint`; all are divided by the artificial strip depth h.

6. The KKT matrix is factorized once; $\mathbf{V}_0$ and $\mathbf{V}_1$ are obtained by two back-substitutions, and `finalize_v1_and_compute_deff` returns the generalized-Timoshenko 6×6, symmetrised.

7. `write_sc_K` writes `<yaml>_Timo.out`, and the returned bundle carries `Timo`, `V0`, `V1`, the contour geometry, the strip, the per-element layup names and the layup and material databases — everything the two-step dehomogenization needs.

The shipped example is `examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section`, whose input is `iea_s10_shell.yaml` (the IEA-22 blade at $r/R = 0.2$):

```
cd examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section
python beam_homo_shell.py
```

It writes `iea_s10_shell_Timo.out` (the Timoshenko stiffness and compliance), `iea_s10_shell_ABDG.out` (the per-section wall laws), `iea_s10_mesh.png` (the contour coloured by layup) and `iea_s10_orient.png` (the compulsory $e_1/e_2/e_3$ orientation figure, emitted by `auto_emit`). The printed diagonal is $[EA, GA_2, GA_3, GJ, EI_2, EI_3]$.

1.4.2 msg-shell: 3-D segment → aperiodic Timoshenko 6×6

The ring pipeline assumes the beam is prismatic: periodic along the axis, so one cross-section suffices. A tapered segment is not periodic, so the periodicity condition has to be replaced. The entry function is `segment_timo_from_3dyaml(seg_yaml, workdir=None, lam_space="elem", shear="full")` and the input is a 3-D shell segment YAML (nodes, quads, sections, materials).

1. `sg_mesh.extract` finds the two end cross-sections *topologically*: a mesh edge used by exactly one quad is a free edge, and the connected components of the free-edge graph are the end rings. Each is written out as a standalone 1-D SG YAML (so you can inspect or re-run it on its own) together with an orientation PNG, and the whole bundle — segment coordinates, cells, subdomains, the three element frame vectors, and the node-to-segment map — is saved to `<base>_boundaries.npz`.

2. Each end ring is solved as a ring in its own right by `ring_indep`, giving `C6`, `V0` and `V1` per side.

3. The full 3-D quad mesh is assembled with the same `assemble_segment_indep` and `assemble_constraint`, with the element hoop curvature `k22_e` and geometric curvature `kg_e` computed from the element centroids and frames. The drilling multipliers are appended as extra rows and columns by `_augment_drilling`, giving a saddle-point system with zero penalty.

4. `_boundary_dirichlet` maps the ring warping fields onto the segment's end nodes. This is the boundary condition that replaces periodicity: $\mathbf{V}_0 = \mathbf{V}_{0,\text{ring}}$ and $\mathbf{V}_1 = \mathbf{V}_{1,\text{ring}}$ on both end sections.

5. `dirichlet_factor` factorizes the constrained interior once; $\mathbf{V}_0$ and then $\mathbf{V}_1$ are solved against it. The Euler–Bernoulli block is $A_{EB} = (\mathbf{D}_{ee} + \mathbf{V}_0^T \mathbf{D}_{he})/L$ with L the axial extent of the nodes, and every energy block handed to `finalize_v1_and_compute_deff` is likewise divided by L, so the result is a per-unit-length stiffness.

6. `<yaml>_Timo.out` is written with the segment length in its banner; the returned dict carries `S6`, `C6L`, `C6R`, `L` and `solve_time`.

Two acceptance checks come with this route. On a *prismatic* segment the segment 6×6 must reproduce its own boundary ring 6×6; measured on an isotropic and a ± 45 tube this identity holds to $\leq 5 \times 10^{-5}$ relative. On a genuinely tapered segment no identity holds, and the check is instead that each diagonal term of the segment lies between the two ring values — the ratio $(S_{ii} - R_{ii})/(L_{ii} - R_{ii})$ printed by the example must fall in $[0, 1]$. The shipped case is a BAR-URC 100 m blade segment:

```
cd examples/OpenSG_shell/5_taper_segment_aperiodic_boundaries
python run_bar_urc_segment.py
```

which reads `BAR_URC_numEl_52_segment_12.yaml` and writes `BAR_URC_numEl_52_segment_12_Timo.out`, `BAR_URC_numEl_52_segment_12_boundaries.npz`, the two boundary YAMLs `boundary_..._L.yaml` and `boundary_..._R.yaml`, three orientation PNGs, and the comparison table `bar_urc_segment12.dat`.

1.4.3 msg-shell: 3-D shell SG → equivalent solid C_{3D}

Here the SG is a surface in three dimensions — a triply periodic minimal surface, a lattice of sheets — and the answer wanted is the 6×6 of the equivalent homogeneous 3-D solid. The entry function is `shell_sg3d(yaml_path, omega=None, drill_pen=1.0e-3, g_source="msg", boundary="periodic")`.

1. The YAML's nodes, elements and `elementOrientations` are read; the element normal e_3 is columns 7 through 9 of the orientation rows. Triangles are accepted by repeating the last node.

2. The single wall law (ABD at the mid-surface reference plus the 2×2 G, again from `rm_plate_msg` when `g_source="msg"`) is built once and applied to every element.

3. Junction lines are counted: any shell edge shared by more than two elements. The count appears in the `.out` banner, because it tells you how much of the cell's stiffness comes from junction physics the plain shell lattice does not resolve.

4. The boundary treatment. With the default `boundary="periodic"` the periodic map ties opposite faces, edges and corners through the sparse assembly map, and the three rigid translations are removed by three area-weighted Lagrange rows. With `boundary="aperiodic"` the macro (affine) field is mapped onto the boundary instead, prescribing zero *translational* fluctuation $w_1 = w_2 = w_3 = 0$ on every node of the bounding-box faces while the rotational DOFs stay free. Clamping the rotations as well over-stiffens the cut edges; freeing them was measured to move C_{11} from +12 to +7 percent. The aperiodic result is a kinematic upper bound on a single cell and converges to the periodic one as the SG grows.

5. The batched Γ_h and Γ_e operators are evaluated at the Gauss points, contracted into the sparse $\mathbf{K}$, $\mathbf{D}_{he}$ and $\mathbf{D}_{ee}$, with a drilling stabilisation of `drill_pen` $\times A_{11}$ on the ω_3 DOF. One `scipy.sparse.linalg.splu` factorization then serves all six macro strain columns.

6. $\mathbf{D}_{\text{eff}} = \mathbf{D}_{ee} + \mathbf{V}_0^T \mathbf{D}_{he}$ is symmetrised and divided by ω. The default ω is the integrated midsurface *surface area* — the 3-D analogue of the plane-section ω = perimeter — and that is the normalisation of the returned `C3D`. The `<yaml>_C3D.out` file is instead written per unit-cell *volume*, so its moduli compare directly with a solid `.K` file.

Because the wall law is a Reissner–Mindlin plate law, the sheet thickness is a parameter of the material model rather than of the mesh: the same surface mesh serves any thickness, which is what makes a thickness sweep cheap. Figure 1.2 shows the Schwarz-P unit cell that the shipped example uses.

```
cd examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG
python make_schwarz_yaml.py
python schwarz_solid_props.py
```

The run prints the junction-edge count, the number of DOF and the solve time, then the 6×6; it writes `schwarz_p_3Dshell_C3D.out` and `schwarz_p_mesh.png`. The companion script `compare_boundary_modes.py` runs the same cell under both boundary treatments and tabulates them in `boundary_compare.dat`. This route is validated against its own solid counterpart: for the Schwarz-P cell the shell SG sits within −3 percent on the normal moduli, −6 percent on the shears and +0.3 percent on Poisson's ratio relative to the SwiftComp-exact msg-solid answer.

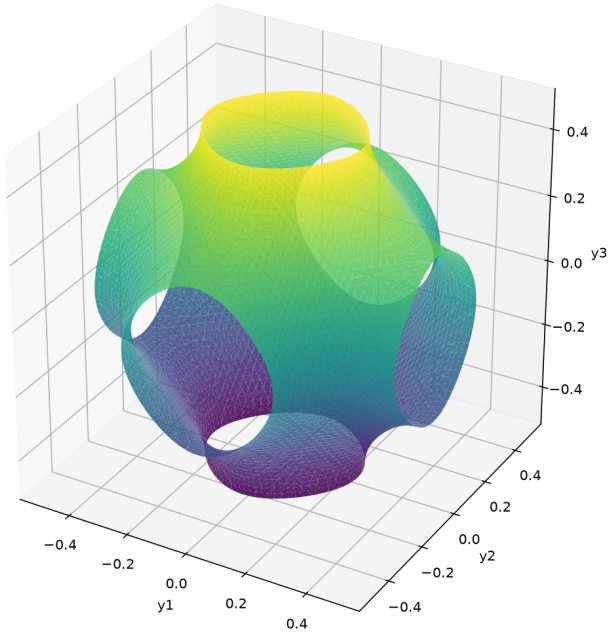

Figure 1.2: The Schwarz-P triply periodic minimal surface meshed as a 3-D shell structure gene. The surface carries a Reissner–Mindlin wall law, so sheet thickness is a material parameter and one mesh serves an entire thickness sweep. Produced by `schwarz_solid_props.py` in `examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG`.

1.4.4 msg-solid: one driver, three macro models

The msg-solid engine has a single public entry,

```
from opensg_solid.sg_homo import plate_homo_2d

r = plate_homo_2d(sc_path,                # a .sc or .yaml SG file
                  material_param=None,    # (n_mat, 9) engineering override
                  angles=None,            # deg per material
                  n_model=2,              # 1 beam, 2 plate, 3 3-D solid
                  workdir=None,
                  elem_rotation=None,     # (E, 9) per-element direction cosines
                  solver="direct",        # "direct" | "cg"
                  shear_refined=False,    # + the RM G 2x2 (plate only)
                  plot=True,              # writes <base>_mesh.png
                  boundary=None)          # "periodic" (default) | "aperiodic"
```

Listing 1.2: The msg-solid driver signature: n˙model selects which macro strain set is built.

and the argument that selects the physics is `n_model`: it decides which macro strain set Γ_e is built. The steps shared by all three are:

1. `sg_mesh.load_sg_input` parses the `.sc` or YAML into a dict carrying `dim` (the SG dimension, 1, 2 or 3), `nodes`, `cells`, `mat_id` and the materials. With `plot=True` it also writes `<base>_mesh.png` of the actual mesh, elements coloured by material. Mixed element types in one SG are rejected — the solver runs one homogeneous batch.

2. `sg_mesh._cell_basis` maps (SG dimension, nodes per element) to a basix cell type and basis degree; `fe_jax.basis_quadrature` then supplies the quadrature points `xi_qp`, weights `W_q`, basis values `phi_qn` and parametric gradients `dphi_dxi_qnp`. The quadrature degree is 6 for higher-order elements and 2 for linear ones.

3. `fe_jax.setup.mesh_to_jax` produces the element-node coordinate array `x_end` of shape (E, N, d), and `mesh_to_periodic_sparse_assembly_map` produces the master connectivity and the DOF map that carries periodicity. Periodicity rides entirely in the assembly map; no transformation matrix is formed. With `boundary="aperiodic"` the map is the identity and instead every bounding-box-face node gets $w = 0$ Dirichlet; that option is available for the plate and solid models with `solver="direct"` only.

4. `sg_materials.get_heterogeneous_C_matrix` builds the per-element 6×6 table `C_ess` of shape $(E, 6, 6)$, applying `angles` and `elem_rotation` (per-element frames are honoured by all three models, not only the beam).

From there the routes diverge:

`n_model=1`**, beam.** Control goes to `_beam_homo_kkt`: the four Euler–Bernoulli macro modes $[\varepsilon_{11}\ \kappa_1\ \kappa_2\ \kappa_3]$ are solved under four rigid-body Lagrange constraints, then the first-order chain runs and the generalized-Timoshenko reduction produces the 6×6. The result dict carries `C_eff` (the Timoshenko 6×6) and `C_eff_EB` (the classical 4×4 from the same run). The output is `<base>.out`, named *Timoshenko*, with ω in the banner. This path needs an SG of dimension ≥ 2.

`n_model=2`**, plate.** The macro strain set is $\bar{\varepsilon} = [\varepsilon_{11}\ \varepsilon_{22}\ 2\varepsilon_{12}\ \kappa_{11}\ \kappa_{22}\ \kappa_{12}]$ and the answer is the plate ABD, written as *Classical Plate*. With `shear_refined=True` the first-order warping ladder `plate_shear_ladder` runs as well and the result gains `G_msg` (2×2), `ABDG` (the 8×8 with the shear block on the diagonal) and `A6_ladder`.

`n_model=3`**, solid.** The macro strain set is the six 3-D strains and the answer is the equivalent solid 6×6, written with the orthotropic-approximated engineering constants appended (E_1, E_2, E_3, G_{12}, G_{13}, G_{23}, ν_{12}, ν_{13}, ν_{23}).

Models 2 and 3 share the assembly and solve code (`_homo_direct`), so switching between a plate law and a solid law on the same mesh is a one-character change. The shipped examples are numbered by capability under `examples/OpenSG-solid/`: `1_get_plate_props_from_1DSG` (a stacking sequence in `layup_db.yaml` → `layup_db_plate_homo.out`, run with `python 1d_sg.py layup_db.yaml`), `3_get_plate_props_from_2D_SG`, `6_get_solid_props_from_3D_SG`, and `7_get_beam_props_from_SG`, whose script is:

```
cd examples/OpenSG-solid/7_get_beam_props_from_SG
python beam_homo_sg.py
```

That one reads `RHC_SW_2UC_45.yaml` (a two-unit-cell honeycomb cross-section) with `n_model = 1` and three materials overridden through `material_param` and `angles`, and writes `RHC_SW_2UC_45.out` plus `RHC_SW_2UC_45_mesh.png`.

1.5 GPU readiness

The msg-solid engine has two interchangeable linear solvers, and the choice is the `solver` argument of `plate_homo_2d`.

`solver="direct"` is the default. The jitted element kernels build the sparse matrix, which is handed as a CSR to `_sparse_direct_solve` — pypardiso (Intel MKL PARDISO) when it is importable, SciPy's SuperLU otherwise. One factorization serves all macro strain columns. This is the fastest route on a workstation, and it is the only stage of the calculation that leaves the accelerator.

`solver="cg"` is the GPU route. The element kernels (`calculate_RHS_and_Ke_batch_periodic` and the jitted `jacfwd` stiffness), the element-by-element operator (`ebe_jacobian_product_periodic`), the block and Chebyshev preconditioners (`compute_block_inv_diag`, `apply_block_precond`, `apply_chebyshev_precond`, `estimate_max_eigenvalue`) and the CG solver itself (`fe_jax.solve_cg`) are pure JAX and run device-resident: no global matrix is ever formed, so memory scales with the element batch rather than with the bandwidth of an assembled operator. On a GPU host, `plate_homo_2d(..., solver="cg")` therefore keeps the entire solve on the device.

Switching solvers is digit-safe. The effective law $\mathbf{C}_{\text{eff}}$ is *stationary* in $\mathbf{V}_0$ — equation (1.4) evaluates the energy at its own minimiser — so an error δ in the fluctuation field enters the answer at $O(\delta^2)$. A solver tolerance loose enough to be visible in $\mathbf{V}_0$ is still invisible in the printed matrix. Both routes are documented to produce the same digits.

The msg-shell routes are one step behind. Their operators are assembled as batched einsum contractions in the two-step form $\mathbf{CB}$ then $\mathbf{B}^T(\mathbf{CB})$, and those batches are written against the NumPy/`jnp` API and are portable to the device; the sparse factorization (SuperLU for the ring and the 3-D shell SG, pypardiso for the KKT solve in `msg_solver`) is the remaining CPU stage. For the SG sizes these routes handle — a cross-section contour is a few thousand DOF — that factorization is not the bottleneck.